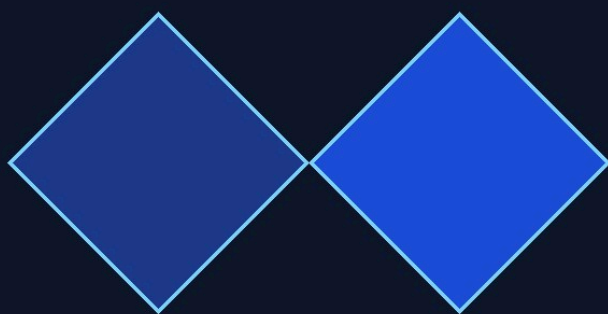




FORMATION C/C++ - LES BASES

C/C++

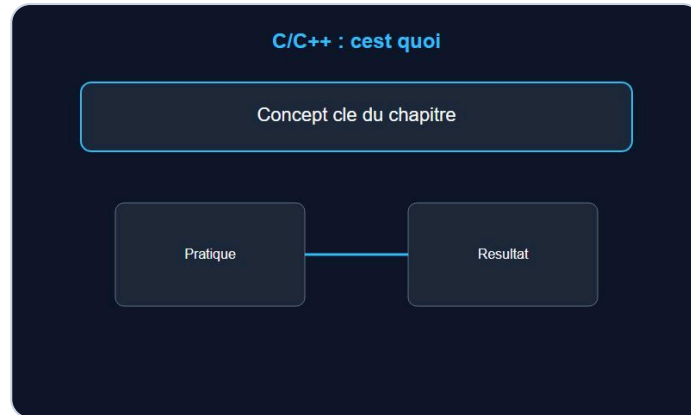
Comprendre la performance, la memoire
et les bases modernes.

DanielCraft - Livre debutant - Pack Systeme

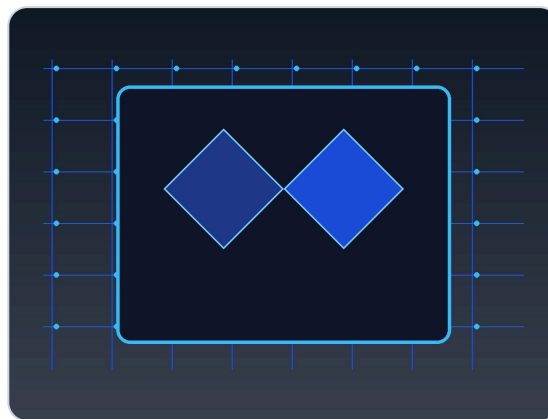
Sommaire

1. [C'est quoi C et C++ ?](#)
 - [En une phrase](#)
 - [Pourquoi apprendre C/C++ ?](#)
2. [Installer l'environnement](#)
 - [Outils essentiels](#)
 - [Verification](#)
3. [Premier programme](#)
4. [Variables et types](#)
5. [Conditions](#)
6. [Boucles](#)
7. [Fonctions](#)
8. [Tableaux et vecteurs](#)
9. [Pointeurs et memoire](#)
10. [Classes et objets](#)
11. [Heritage et polymorphisme](#)
12. [Gestion d'erreurs](#)
13. [Mini-projet : gestionnaire de taches CLI](#)
14. [Ce qu'il faut retenir](#)
15. [Atelier : variables et calculs](#)
16. [Atelier : fonctions](#)
17. [Atelier : classes](#)
18. [Erreurs classiques](#)
19. [Bonnes pratiques](#)
20. [Quiz C/C++ - Les bases](#)
21. [Bravo !](#)

C'est quoi C et C++ ?



C/C++ : performance et controle.



Univers C/C++.

En une phrase

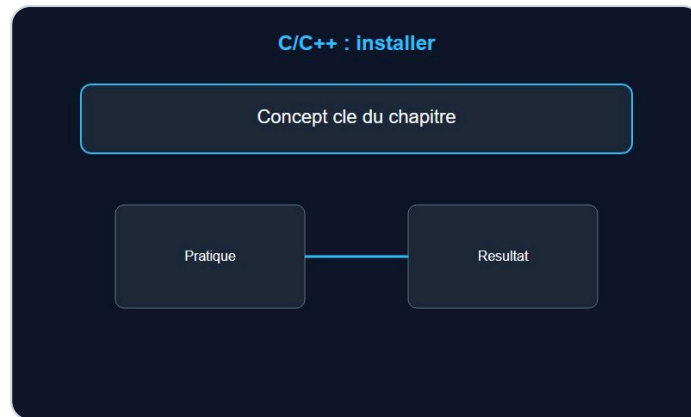
C est un langage systeme rapide et proche du materiel. C++ ajoute la programmation orientee objet, generique et moderne.

Pourquoi apprendre C/C++ ?

- Comprendre la memoire et les performances.
- Ecrire des applis natives (logiciels, jeux, embarque).
- Lire et maintenir de tres nombreux projets industriels.

> **Astuce DanielCraft** - C t'apprend la machine. C++ t'apprend a construire grand proprement.

Installer l'environnement



Compiler + CMake + IDE.

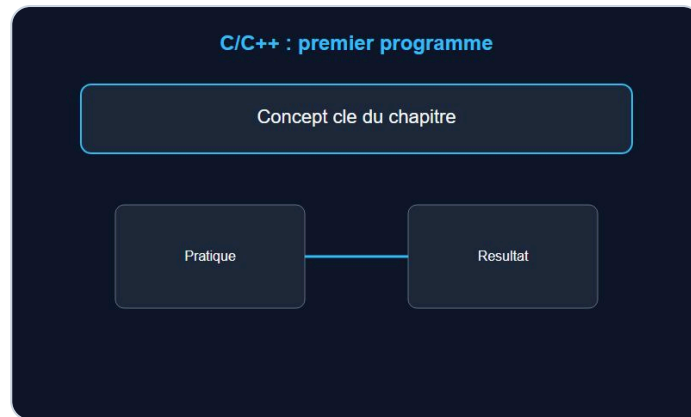
Outils essentiels

- **Compilateur** : GCC, Clang, ou MSVC.
- **Build** : CMake.
- **IDE** : VS Code + extension C/C++.

Vérification

```
g++ --version  
cmake --version
```

Premier programme



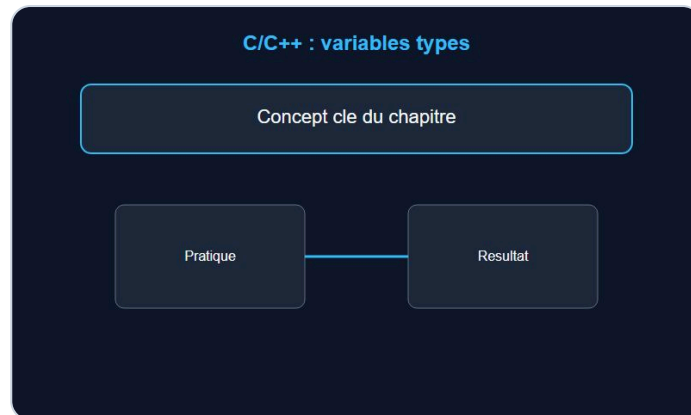
Premier programme C++.

```
#include <iostream>
int main() {
    std::cout << "Bonjour C++ !\n";
    return 0;
}
```

Compiler :

```
g++ main.cpp -o app
./app
```

Variables et types

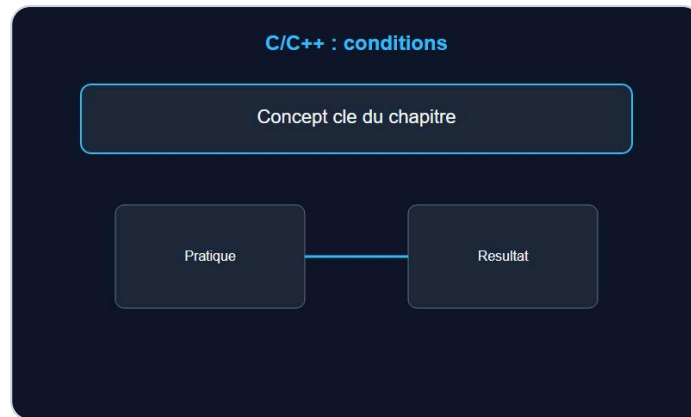


Types et variables.

- Entiers : `int`, `long`, `short`
- Flottants : `float`, `double`
- Texte : `char`, `std::string`
- Booleen : `bool`

```
int age = 25;
double prix = 19.99;
std::string nom = "Nora";
```

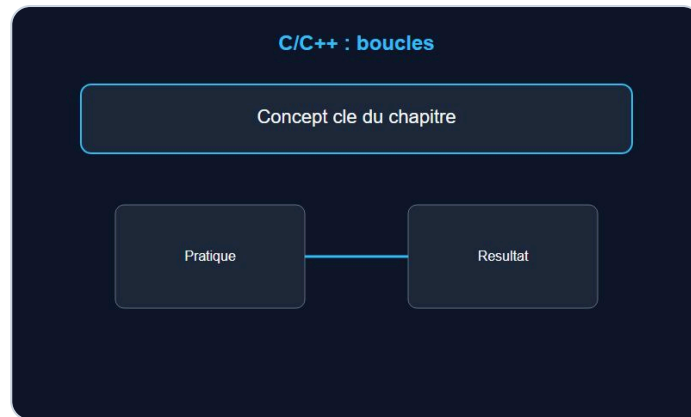
Conditions



Conditions if/else.

```
if (note >= 16) {  
    std::cout << "Tres bien\n";  
} else if (note >= 10) {  
    std::cout << "Valide\n";  
} else {  
    std::cout << "A retravailler\n";  
}
```

Boucles



Boucles for/while.

```
for (int i = 0; i < 5; ++i) std::cout << i << "\n";

int n = 0;
while (n < 3) { ++n; }
```


Fonctions



Coder en C++ au quotidien.



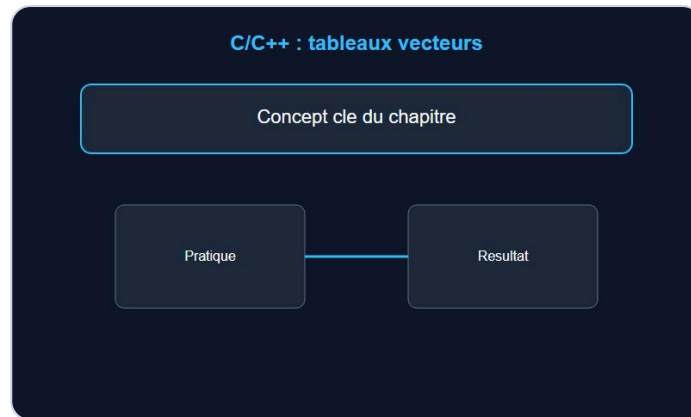
Fonctions et references.

```
int addition(int a, int b) {  
    return a + b;  
}
```

Passe par reference pour eviter des copies :

```
void afficher(const std::string& nom) {  
    std::cout << nom << "\n";  
}
```

Tableaux et vecteurs

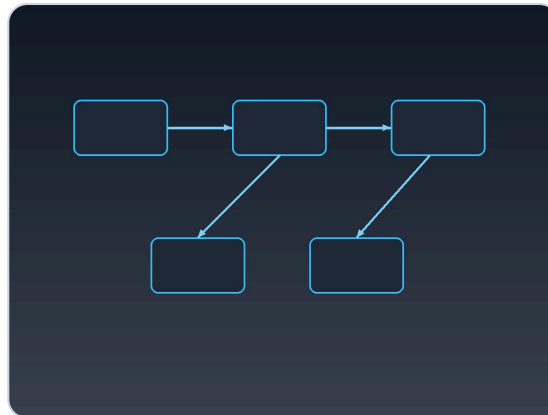


Tableaux et vecteurs STL.

```
int notes[3] = {12, 15, 18};  
std::vector<int> points = {1, 2, 3, 4};  
points.push_back(5);
```

`std::vector` est prefere dans la plupart des cas.

Pointeurs et memoire



Visualiser les pointeurs.



Pointeurs et memoire.

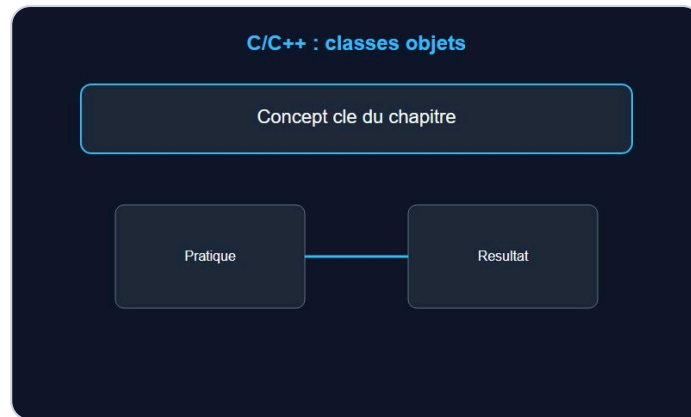
```
int x = 42;
int* p = &x;
std::cout << *p << "\n";
```

Allocation dynamique :

```
int* tab = new int[10];
delete[] tab;
```

> **Piege** - Oublier `delete` provoque des fuites memoire.

Classes et objets



Classes et objets.

```
class Compte {  
private:  
    double solde;  
public:  
    Compte(double s) : solde(s) {}  
    void depot(double m) { solde += m; }  
    double lire() const { return solde; }  
};
```

Heritage et polymorphisme

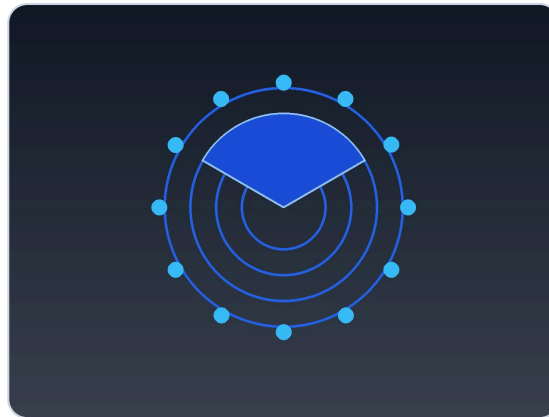


Heritage et polymorphisme.

```
class Animal {
public:
    virtual void parler() const { std::cout << "... " << "\n"; }
    virtual ~Animal() = default;
};

class Chien : public Animal {
public:
    void parler() const override { std::cout << "Wouf" << "\n"; }
};
```

Gestion d'erreurs



Optimisation et performance.



Gestion d'erreurs C++.

```

try {
    throw std::runtime_error("Probleme");
} catch (const std::exception& e) {
    std::cout << e.what() << "\n";
}
    
```

En C moderne, on combine exceptions, validation d'entrees et assertions.

Mini-projet : gestionnaire de tâches CLI



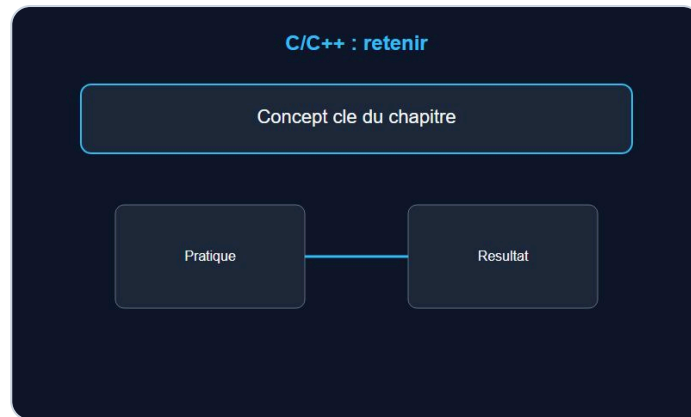
Mini-projet CLI.

Objectif : ajouter, lister et supprimer des tâches avec `std::vector<std::string>`.

Compétences pratiquées :

- boucles,
- fonctions,
- vecteurs,
- persistance simple fichier.

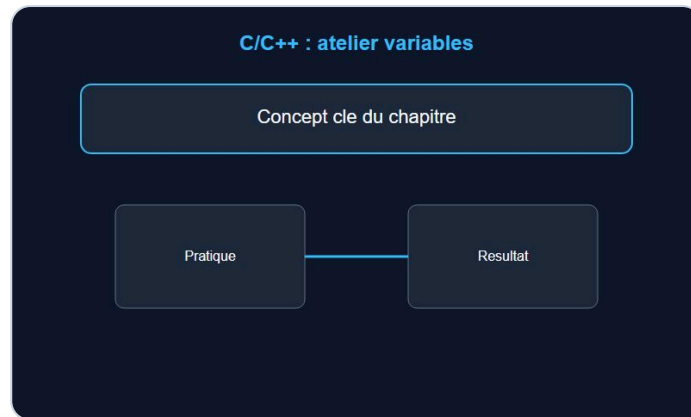
Ce qu'il faut retenir



Carte des notions C/C++.

- C/C++ = performance et controle.
- References et pointeurs sont fondamentaux.
- `std::vector` et `std::string` simplifient le quotidien.
- Le destructeur et RAII evitent les fuites memoire.

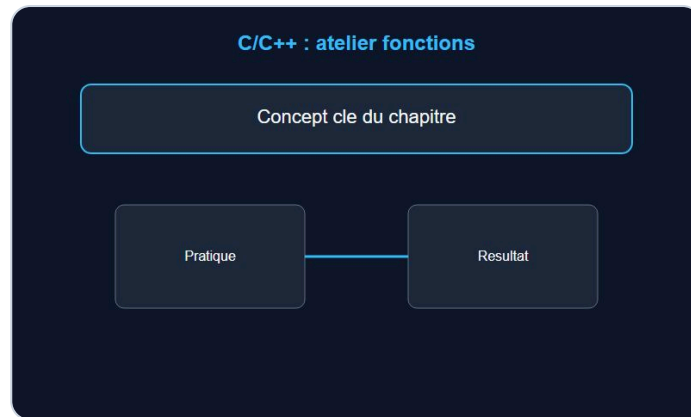
Atelier : variables et calculs



Atelier variables.

1. Lire 3 notes et calculer la moyenne.
2. Convertir des degres Celsius en Fahrenheit.
3. Afficher min, max, somme.

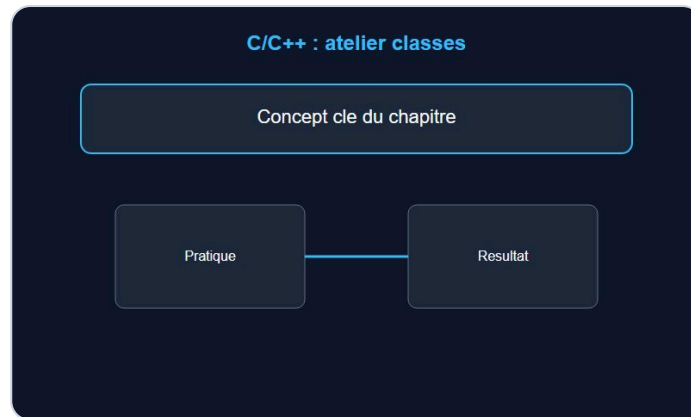
Atelier : fonctions



Atelier fonctions.

1. 1. Ecrire `int max2(int a, int b)`.
2. 2. Ecrire `double moyenne(const std::vector<int>& v)`.
3. 3. Ecrire une fonction de recherche lineaire.

Atelier : classes

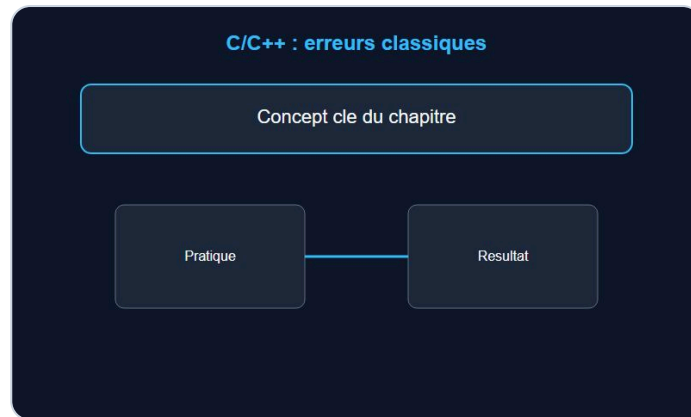


Atelier classes.

Créer une classe `Produit` avec :

- `nom`, `prix`,
- constructeur,
- méthode `afficher()`,
- méthode `appliquer_remise(double p)`.

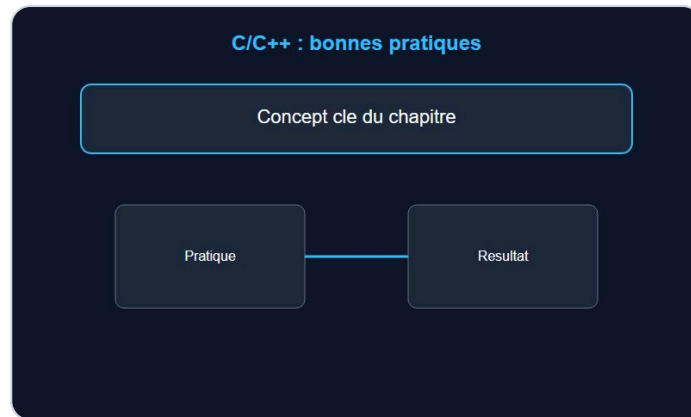
Erreurs classiques



Erreurs classiques.

- Oublier `;`
- Confondre `=` et `==`
- Sortir des limites d'un tableau
- Fuites memoire avec `new` / `delete`
- Copier inutilement de gros objets

Bonnes pratiques

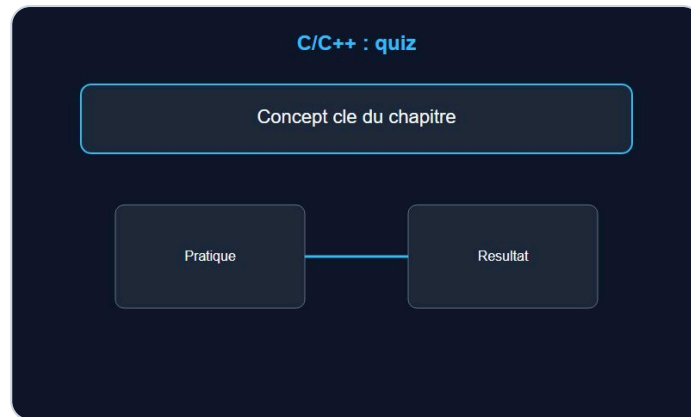


Bonnes pratiques.

- Activer les warnings : `-Wall -Wextra -Wpedantic`
- Preferer `std::vector` aux tableaux bruts
- Preferer RAII et smart pointers (`std::unique_ptr`)
- Formatter et tester regulierement

QUIZ

Quiz C/C++ - Les bases



Quiz de revision.

1. 1. Difference entre pointeur et reference ?
2. 2. Pourquoi `const std::string&` est frequent ?
3. 3. A quoi sert `virtual` ?
4. 4. Quand utiliser `std::vector` ?
5. 5. Que fait un destructeur ?

FINAL

Bravo !



Félicitations !

Tu maîtrises les fondamentaux C/C++ :

- syntaxe de base,
- fonctions,
- mémoire,
- POO,
- mini-projet CLI.

Prochaine étape : C++ intermédiaire (templates, STL avancée, CMake multi-modules, tests).

Tu as les briques. Maintenant, code.